# Лекция 3. Apache Spark и PySpark. Распределённая обработка данных

Технологии и инструментарий анализа больших данных

Курс ТИАБД



Раздел

# Постановка задачи



# Тема третьей лекции

Spark а как **модель распределённого выполнения программы**.

**Главный вопрос**

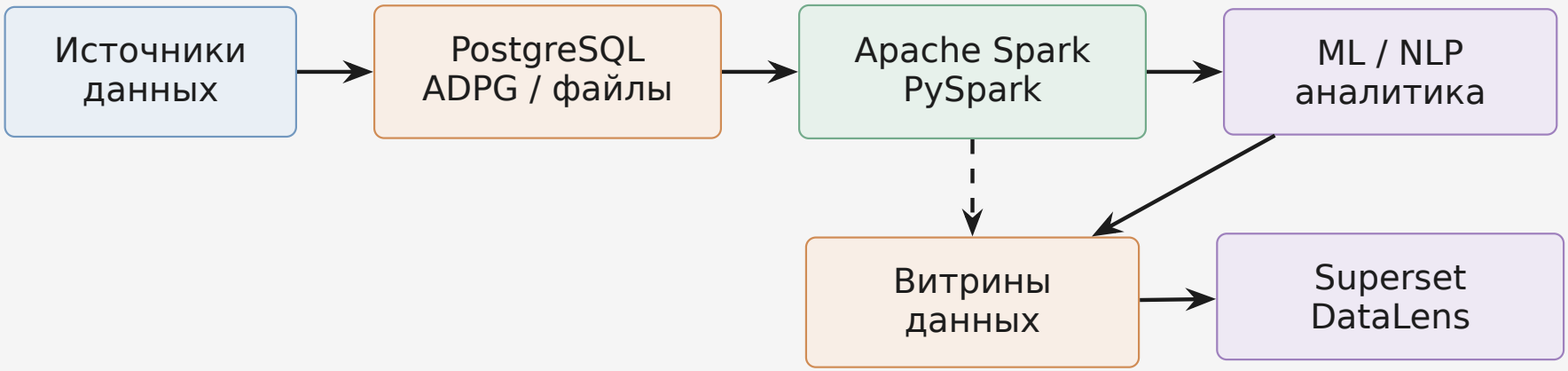
Как логическая операция над таблицей превращается в набор задач, выполняемых параллельно над разными частями данных?

**Ключевые понятия:**

- Драйвер, Исполнитель, Менеджер кластера;
- Spark-сессия, DataFrame, схема;
- трансформации и действия;
- ленивые вычисления, DAG, Job, стадия, задача;
- партиция, внутренние/широкие трансформации, перемешивание;
- join-стратегии, Parquet-оптимизации.



# Место Spark в архитектуре курса




Spark занимает вычислительный слой между хранением данных и прикладной аналитикой. Его назначение — параллельно читать, преобразовывать, агрегировать и подготавливать данные для моделей и BI.



# Когда Pandas перестаёт быть достаточным

Однопроцессная аналитика удобна, пока данные и время обработки соответствуют ресурсам одного компьютера.

**Ограничение памяти**

Если набор данных должен быть представлен в памяти процесса, критическим становится условие

$$|D| > M_{available}.$$

**Ограничение времени**

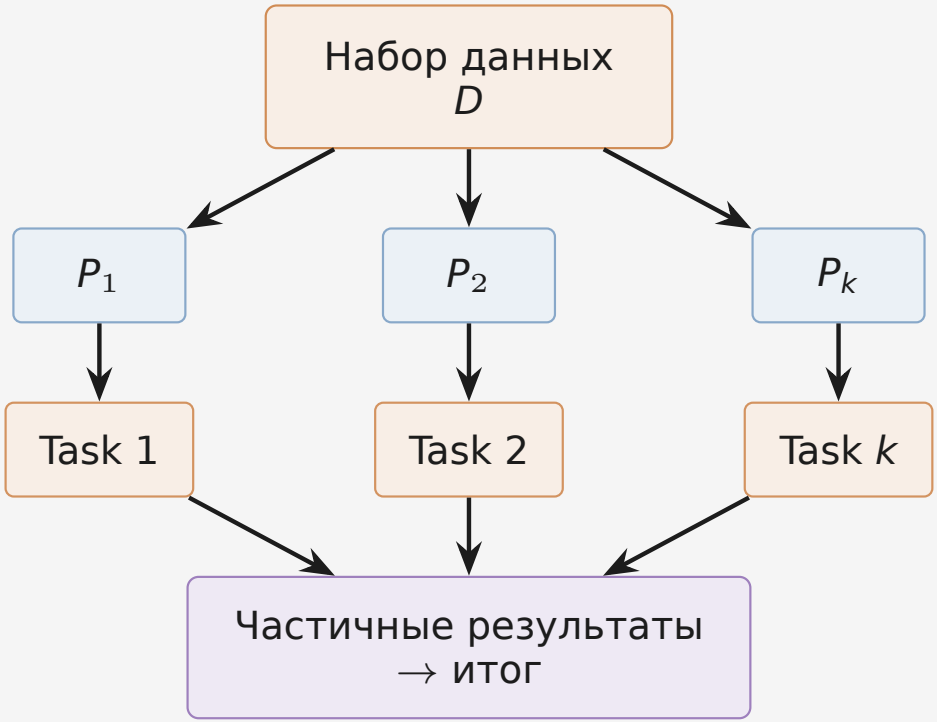
Даже помещающиеся данные могут обрабатываться слишком долго:

$$T_{processing} \gg T_{acceptable}.$$

Большие данные — это несоответствие требуемой обработки возможностям одной машины, одного процесса или одного последовательного алгоритма.



# Переход к распределённой обработке




$$D = P_1 \cup P_2 \cup \ldots \cup P_k.$$

*Распределённая обработка начинается не с синтаксиса API, а с вопроса* ***как разбить работу на части и как затем корректно собрать результат****.*



# Зачем нужен фреймворк

Распределённая система должна решить несколько вопросов.

- Как физически прочитать данные из множества файлов или разделов?
- Как назначить вычислительные задачи исполнителям?
- Как передать код к данным, а не данные к одному процессу?
- Когда можно обрабатывать партиции независимо?
- Когда требуется перераспределение данных между исполнителями?
- Как восстановиться после сбоя отдельной задачи?
- Как избежать лишнего чтения, сортировок и сетевого обмена?

**Определение**

**Apache Spark** — вычислительный движок, который строит план распределённой обработки данных и исполняет его в виде параллельных задач над партициями данных.



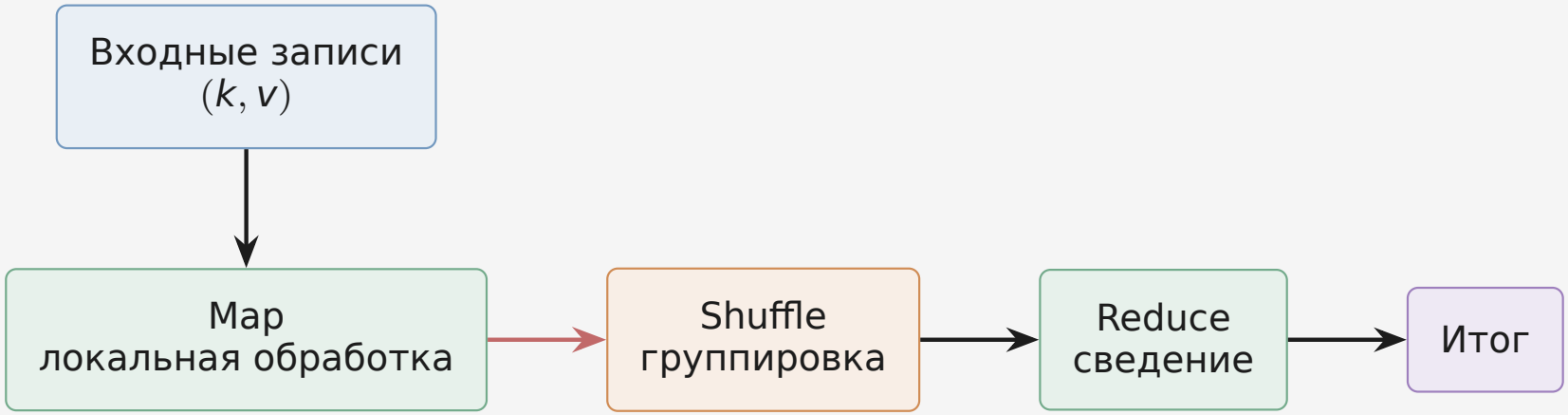
Раздел

# От MapReduce к Spark



# Историческая идея MapReduce

MapReduce полезен как минимальная модель распределённого мышления.




$$Map : (k, v) \to [(k_1, v_1), \ldots, (k_m, v_m)]$$

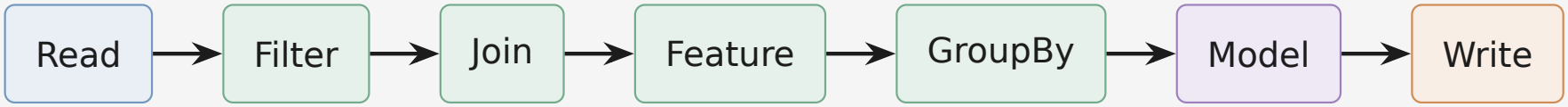
$$Reduce : (k, [v_1, \ldots, v_n]) \to result_k.$$

Сначала выполняется независимая локальная обработка, затем значения с одинаковыми ключами собираются вместе.



# Почему одной модели MapReduce мало для аналитики

В реальной аналитической задаче редко достаточно одной пары Map $\rightarrow$ Reduce. Обычно требуется длинная цепочка операций.

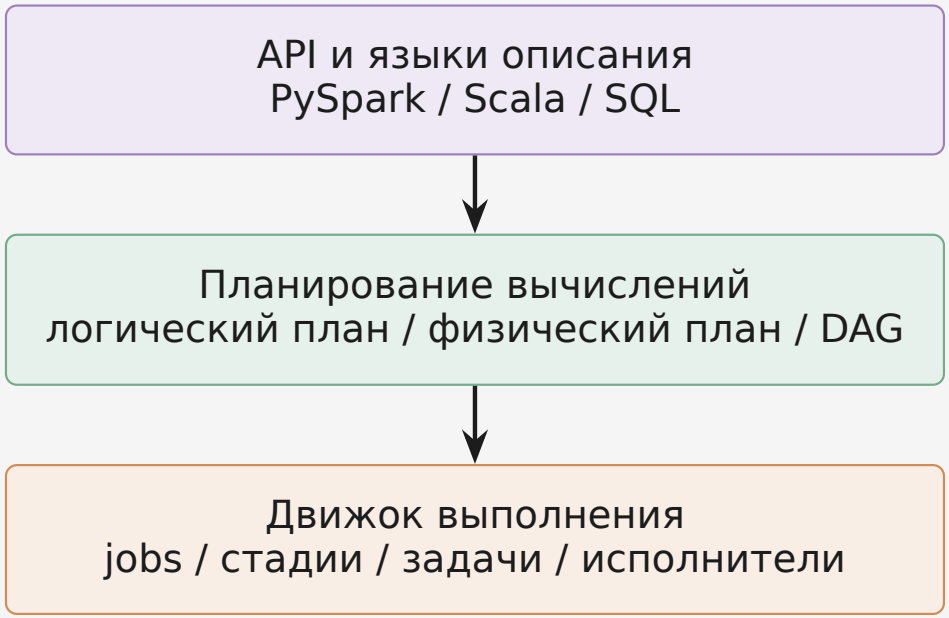



**Как Spark решает проблему MapReduce**

Spark удобен для аналитики потому, что позволяет описывать не отдельную операцию, а **граф вычислений**, включающий чтение, фильтрацию, соединения, агрегации, подготовку признаков и запись результата.

# Spark как три уровня одновременно

**DataFrame**-код в PySpark обычно не означает цикл Python по строкам, классически он описывает план, исполняемый Spark декларативным образом.




- **Apache Spark** — платформа и механизм выполнения.
- **PySpark** — Python-интерфейс к Spark.

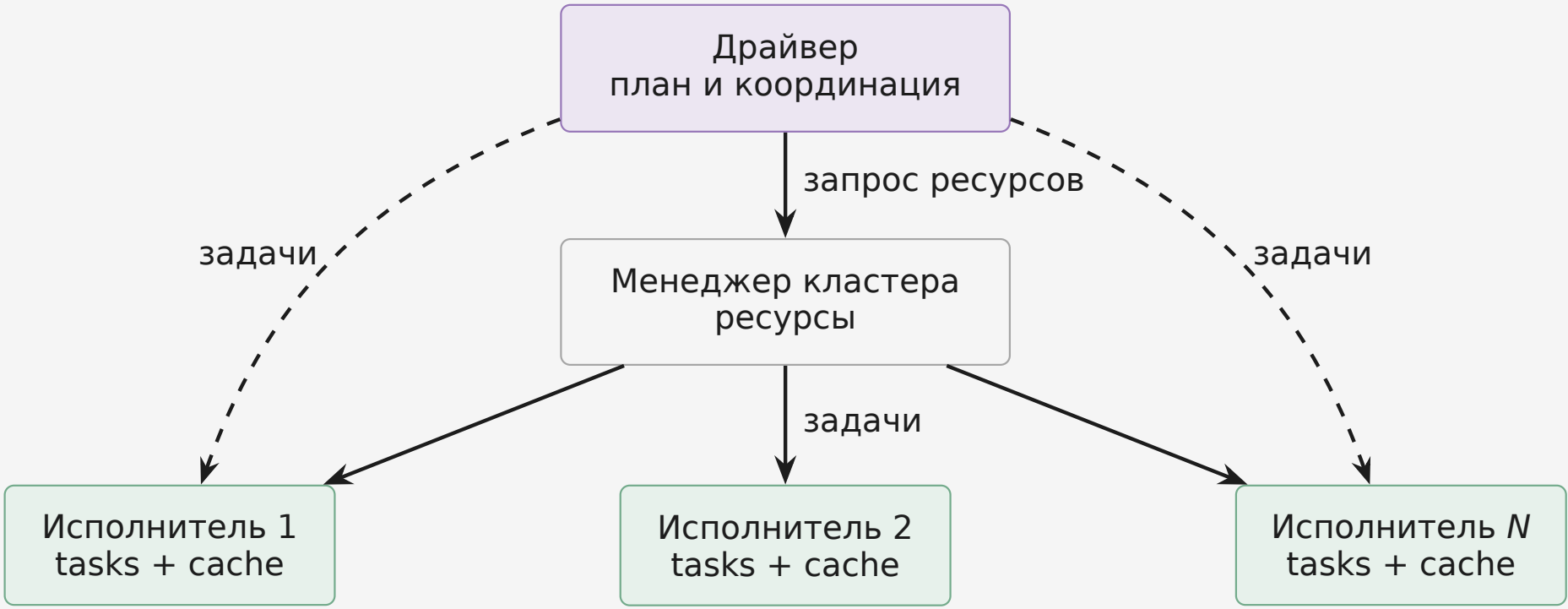


Раздел

# Архитектура Spark-приложения



# Основные роли. Драйвер, Исполнитель, Менеджер кластера




*Драйвер управляет приложением; исполнители выполняют задачи над партициями; менеджер кластера выделяет ресурсы.*



# Драйвер. Управляющий процесс

**Определение**

**Драйвер** — процесс Spark-приложения, в котором создаётся Spark-сессия, формируется план вычислений, запускаются действия и координируется выполнение jobs.

**Функции Драйвера:**

- хранит управляющую логику приложения;
- анализирует цепочку трансформаций;
- инициирует jobs при появлении действий;
- разбивает работу на стадии и задачи;
- получает служебные результаты от исполнителей.

**Инженерное предупреждение**

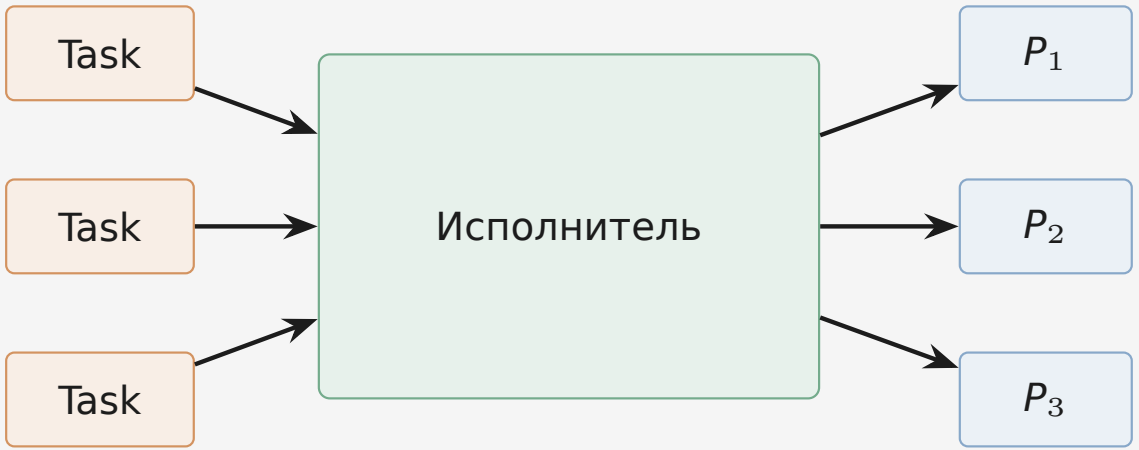
Операция `collect()` собирает данные на ***драйвере***. Для большого результата это может уничтожить преимущество распределённой модели и привести к нехватке памяти в управляющем процессе.



# Исполнитель. Вычислительный процесс

**Определение**

**Исполнитель** — процесс, который выполняет задачи над назначенными партициями и может хранить промежуточные данные в памяти или на диске.




**Смысл.** Исполнители не знают всю программу как ***драйвер***. Они получают конкретные задачи и выполняют их над конкретными частями данных.



# Что означает режим `local[*]`

**Локальный запуск**

Все компоненты физически работают на одном компьютере, используя доступные ядра процессора.

- удобно для обучения;
- не требует кластера;
- сохраняет Spark-модель вычислений.

**Но это не кластер**

`local[*]` не увеличивает RAM, не создаёт сетевое хранилище и не снимает ограничения одного устройства.

- партиции остаются;
- задачи остаются;
- jobs и стадии остаются;
- перемешивание данных можно наблюдать локально.



# Spark-сессия как точка входа

**Определение**

**Spark-сессия** — главный объект современного Spark-приложения, через который выполняются чтение данных, создание DataFrame, SQL-запросы и доступ к конфигурации.

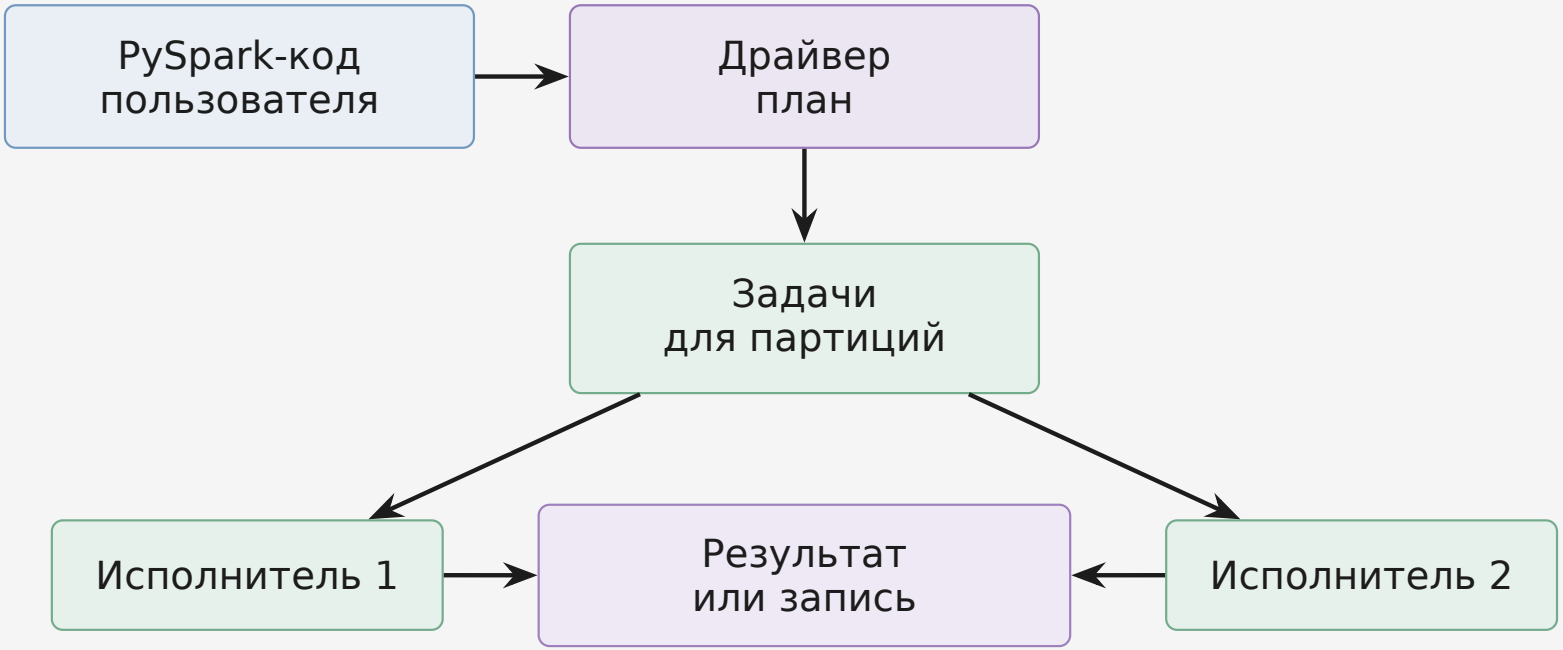
```
from pyspark.sql import SparkSession

spark = (
   SparkSession.builder
   .master("local[*]")
   .appName("store_analysis")
   .getOrCreate()
)
```

**Важно.** Уже в локальном режиме программа строит Spark-план, а не просто выполняет операции как обычный Python-скрипт над списками строк.



# Поток управления в Spark-приложении




*Пользователь описывает вычисление; драйвер планирует как это вычисление будет производиться; исполнители выполняют вычисления; результат возвращается только там, где это действительно требуется.*



Раздел

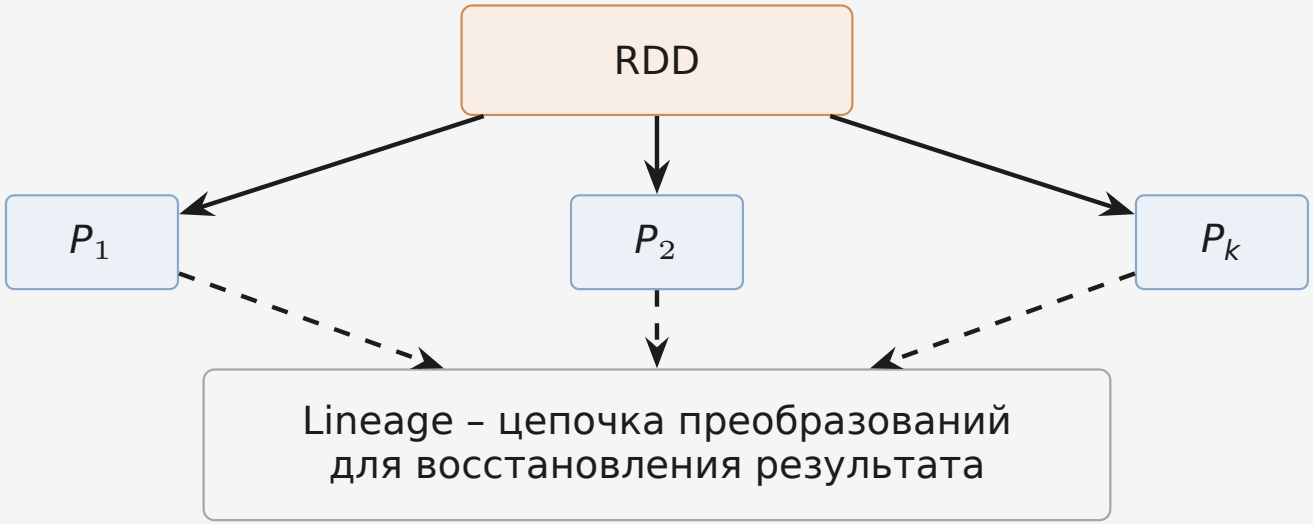
# Модель данных Spark



# RDD. Историческая базовая абстракция

## Определение

**RDD** — Resilient Distributed Dataset, распределённая отказоустойчивая коллекция элементов, разбитая на партиции.



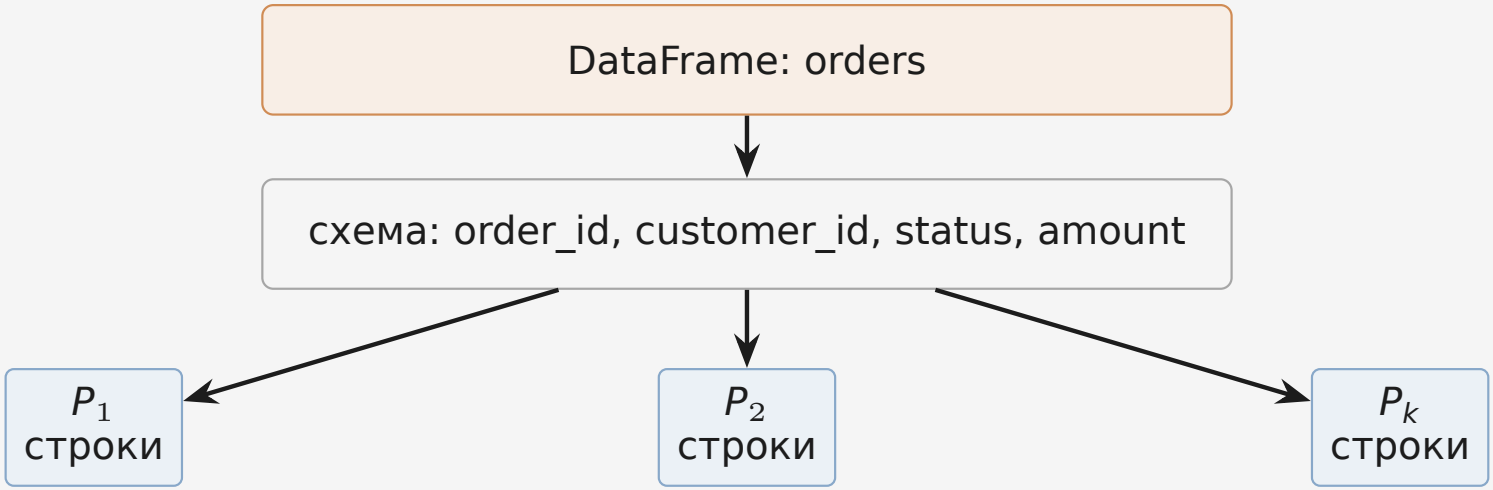

RDD объясняет происхождение понятий партиции, lineage и отказоустойчивости, но основная аналитическая работа будет выполняться через DataFrame.



# DataFrame. Распределённая таблица со схемой

**Определение**

**Spark DataFrame** — логическое представление распределённого табличного набора данных со схемой столбцов и типами данных.




Spark DataFrame не обязан целиком находиться в памяти Python-процесса. Он описывает распределённый набор данных и план операций над ним.

# Зачем Spark нужна схема

Схема связывает имя столбца с типом данных и признаком допустимости пропусков.

| Столбец | Тип | Смысл |
|---|---|---|
| `order_id` | string | идентификатор заказа |
| `customer_id` | string | идентификатор клиента |
| `order_date` | timestamp | время заказа |
| `quantity` | integer | количество |
| `price` | double | цена |

- схема делает поток вычислений воспроизводимым;
- типы позволяют строить корректный план вычислений;
- явная схема снижает риск ошибочного чтения CSV/JSON;
- нарушение схемы становится диагностируемой ошибкой данных.



# Автоматическая и явная схема

## Автоматический вывод

```
df = (
   spark.read
   .option("header", True)
   .option("inferSchema", True)
   .csv("orders.csv")
)
```

Удобно для разведочного анализа, но менее предсказуемо на больших данных.

## Явный контракт

```
schema = StructType([
  StructField("order_id",
       StringType(), False),
  StructField("quantity",
       IntegerType(), True),
  StructField("price", DoubleType()
       , True)
])

df = spark.read.schema(schema).csv(
     "orders.csv")
```

Надёжнее для регулярного pipeline.



# RDD и DataFrame. Различие уровней

| Критерий | RDD | DataFrame |
|---|---|---|
| Уровень | распределённая коллекция | распределённая таблица |
| Структура | элементы произвольного типа | столбцы и schema |
| Оптимизация | меньше информации о смысле операций | больше возможностей для оптимизатора |

**Тезис**

DataFrame даёт Spark сведения о столбцах, фильтрах, соединениях и агрегатах. Поэтому система может оптимизировать не отдельную функцию, а весь план обработки.



Раздел

# Трансформации, действия и ленивые вычисления



# Трансформации и действия

## Трансформации

Описывают новый DataFrame, но не требуют немедленного результата.

- `select`
- `filter / where`
- `withColumn`
- `groupBy`
- `join`

## Действия

Требуют получить наблюдаемый результат и запускают выполнение.

- `count`
- `show`
- `collect`
- `first / take`
- `write`



# Пример цепочки трансформаций

```
from pyspark.sql import functions as F

paid = df.filter(F.col("status") == "paid")

prepared = paid.withColumn(
   "line_amount",
   F.col("quantity") * F.col("price")
)

summary = (
   prepared
   .groupBy("category")
   .agg(F.sum("line_amount").alias("revenue"))
)
```

**Что произошло?**

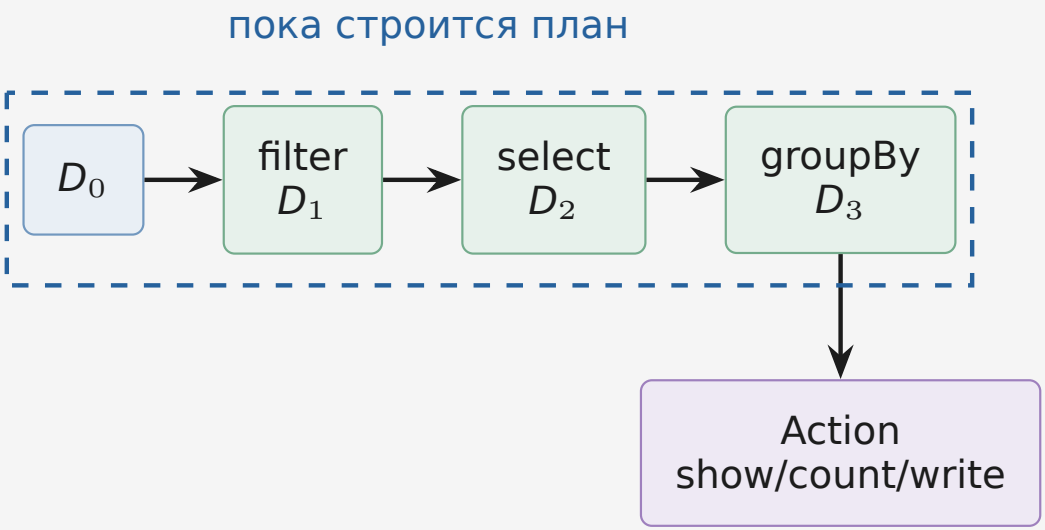
Создано описание результата, но Spark ещё не обязан был читать все строки и считать все суммы. Выполнение начнётся при запуске действия.



# Ленивые вычисления

**Определение**

**Ленивые вычисления** — отложенное выполнение, в котором преобразования накапливаются в вычислительном плане, а реальная обработка начинается только при действиях.




*Ленивость нужна для оптимизации всей цепочки, а не отдельной строки программы.*

# Действие как момент запуска

В приведенном ниже примере Spark до show() в основном строит план. И только после sum начинаются вычисления.

```
result = (
   df
   .filter("status = 'paid'")
   .select("category", "quantity", "price")
   .withColumn("amount", F.col("quantity") * F.col("price"))
   .groupBy("category")
   .agg(F.sum("amount").alias("revenue"))
)

result.show()
```

## Практическая интерпретация

Если программа работает медленно, нужно смотреть не только на Python-код, но и на физический план выполнения Spark.



# Логический и физический план




**Логический план** описывает смысл операций – чтение, фильтрация, проекция, агрегация.
**Физический план** описывает способ выполнения – число стадий, перемешивание, типы соединений, чтение столбцов, обмен данными.

**Команда диагностики**

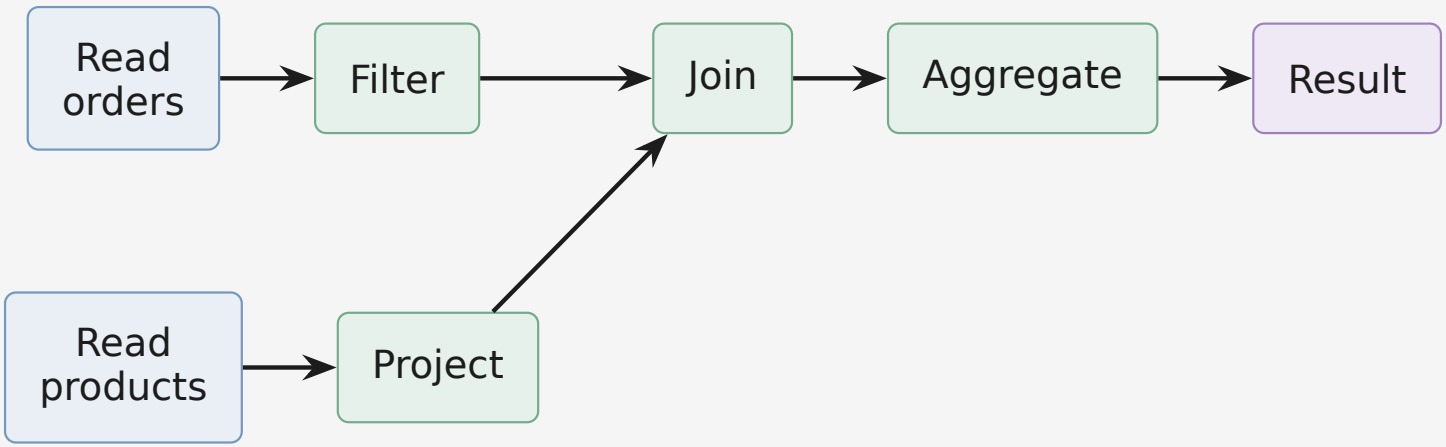
`df.explain()` показывает план выполнения и помогает увидеть, где Spark будет реально перемещать и группировать данные.



# DAG. Граф вычислений

**Определение**

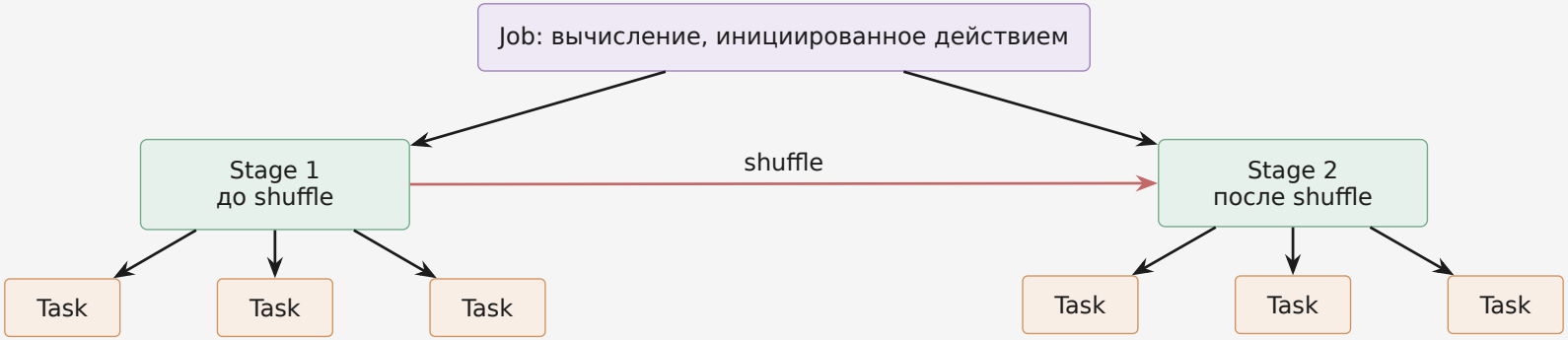
**DAG** — directed acyclic graph, ориентированный ациклический граф зависимостей между операциями вычисления.




*Spark планирует не линейный список команд, а граф зависимостей, который затем разбивается на стадии и задачи.*



# Job, стадия, задача




Job → Стадия → Задача

Раздел

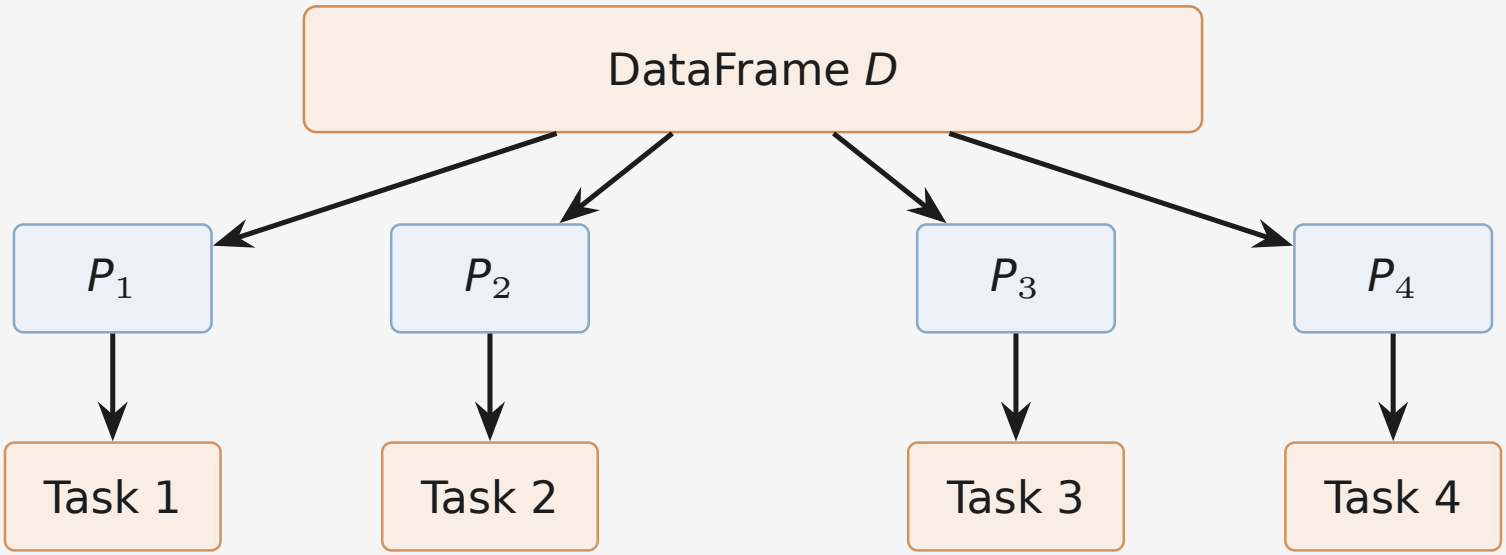
# Партиции и перемешивание



# Партиции как единица параллельной обработки

**Определение**

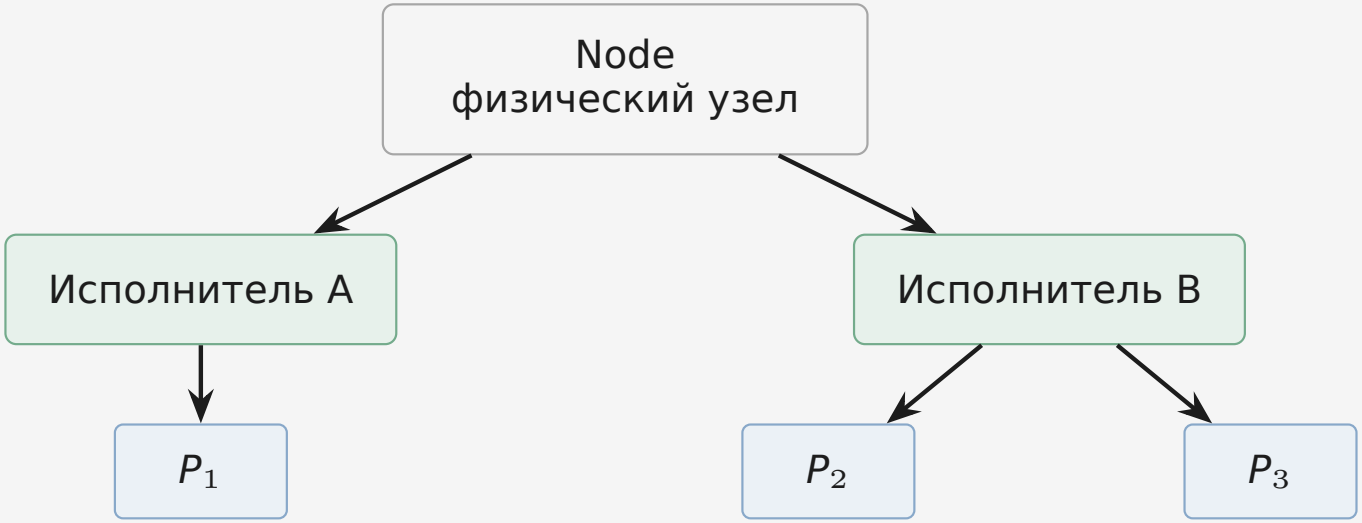
**Partition** — логическая часть распределённого набора данных, обрабатываемая отдельной задачей в рамках конкретной стадии.




**Следствие.** Число партиций ограничивает доступный параллелизм, но слишком большое число мелких партиций увеличивает накладные расходы.



# Партиция, исполнитель и физический узел — разные сущности




## Разделение уровней

$$\text{Партиция} \neq \text{Исполнитель} \neq \text{Узел}.$$

Один Исполнитель может последовательно обработать много задач, а физический узел может содержать несколько процессов выполнения.



# repartition() и coalesce()

## repartition()

Полное перераспределение данных, часто через shuffle.

```
df2 = df.repartition(8)

by_key = df.repartition(
   8, "customer_id"
)
```

## coalesce()

Обычно уменьшение числа partitions с меньшими затратами.

```
df2 = df.coalesce(4)
```

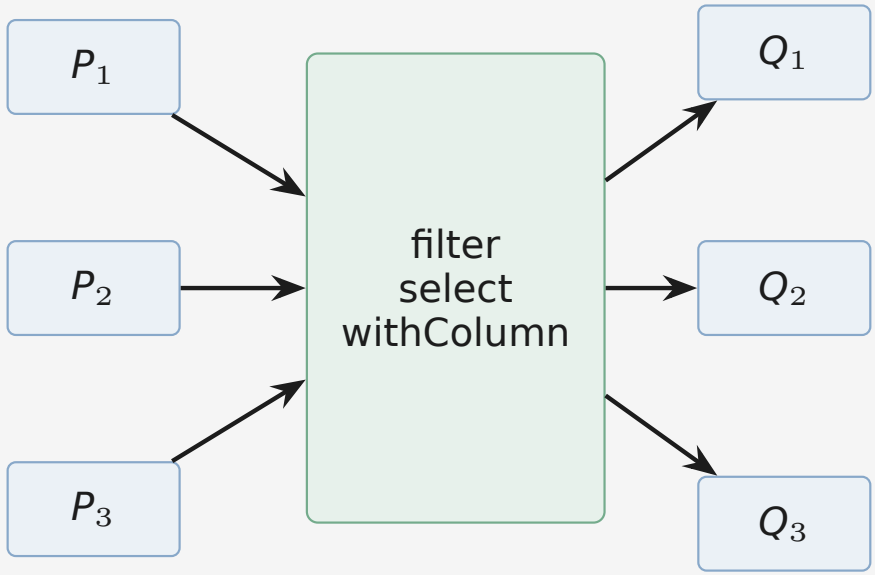
**Эвристика.** `repartition()` полезен для выравнивания и увеличения параллелизма, `coalesce()` — для уменьшения числа partitions перед записью или дальнейшей обработкой.



# Внутренние трансформации

## Определение

**Внутренняя трансформация** — преобразование, при котором выходная партиция может быть вычислена без глобального перераспределения данных между всеми партициями.



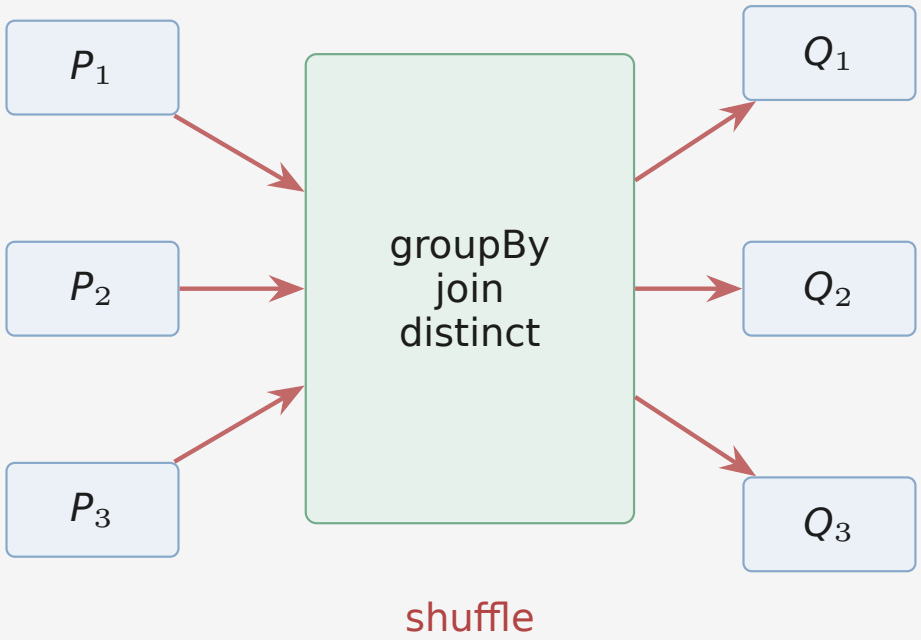

**Примеры:** фильтрация строк, выбор столбцов, вычисление нового столбца по значениям той же строки.



# Широкая трансформация

## Определение

**Широкая трансформация** — преобразование, при котором записи должны быть перераспределены между партициями, например по ключу группировки или соединения.

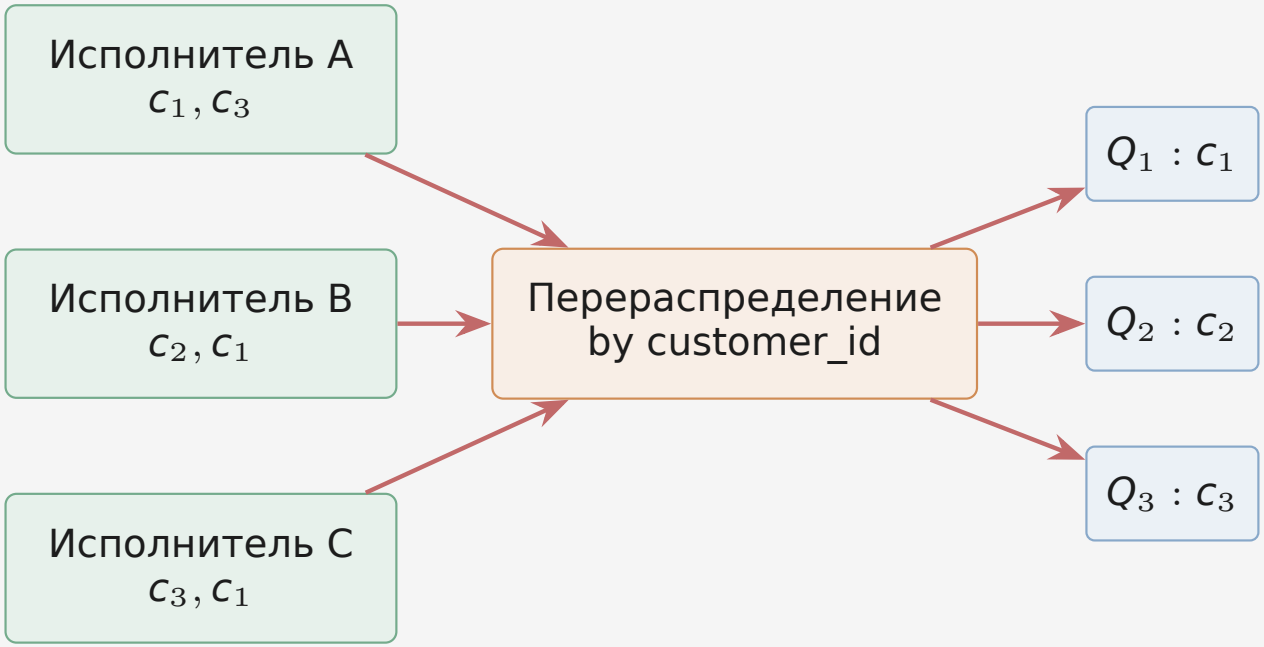



**Главная стоимость** часто связана не с арифметикой, а с перемещением данных.

# Shuffle – перемешивание данных

**Определение**

**Shuffle** — физическое перераспределение записей между партициями, необходимое для wide-преобразований.

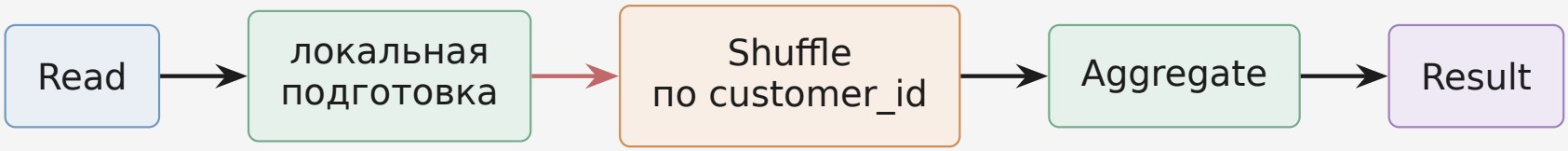



**Shuffle может включать:** сериализацию, запись промежуточных блоков, сетевой обмен, чтение на стороне получателя и сортировку/хеширование.



# Пример. Агрегация по клиенту

```
summary = (
    df
    .groupBy("customer_id")
    .agg(F.sum("amount").alias("total_amount"))
)
```

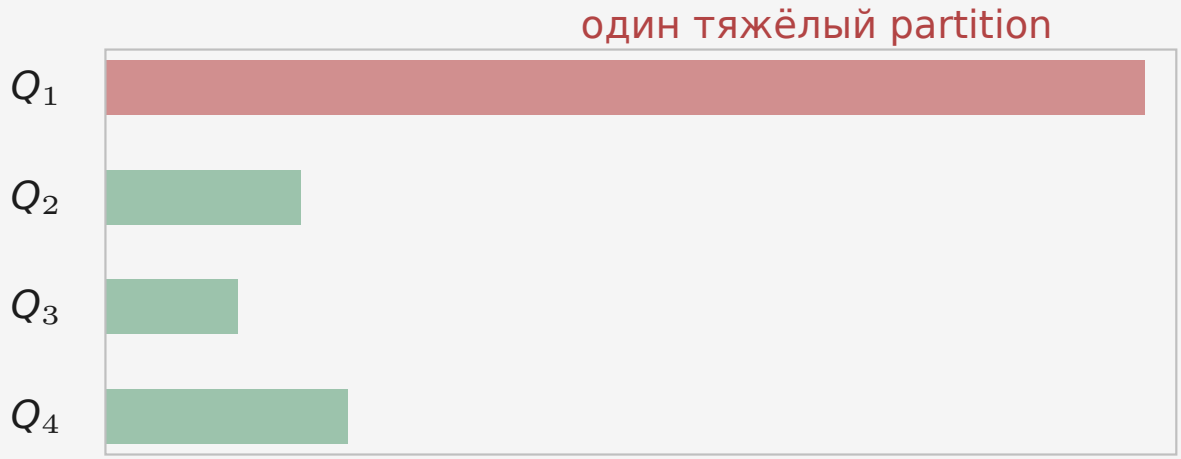



**Тезис**

Сложить числа обычно дёшево. Дорого — собрать все строки одного клиента в нужной группировке.

# Data skew и перекос распределения ключей

Даже при большом числе исполнителей одна стадия может выполняться медленно, если одна партиция получает непропорционально много данных.




$$T_{stage} \approx \max_i T(P_i).$$

Общее время стадии определяется самой медленной партицией, а не средним временем по всем партициям.

Раздел

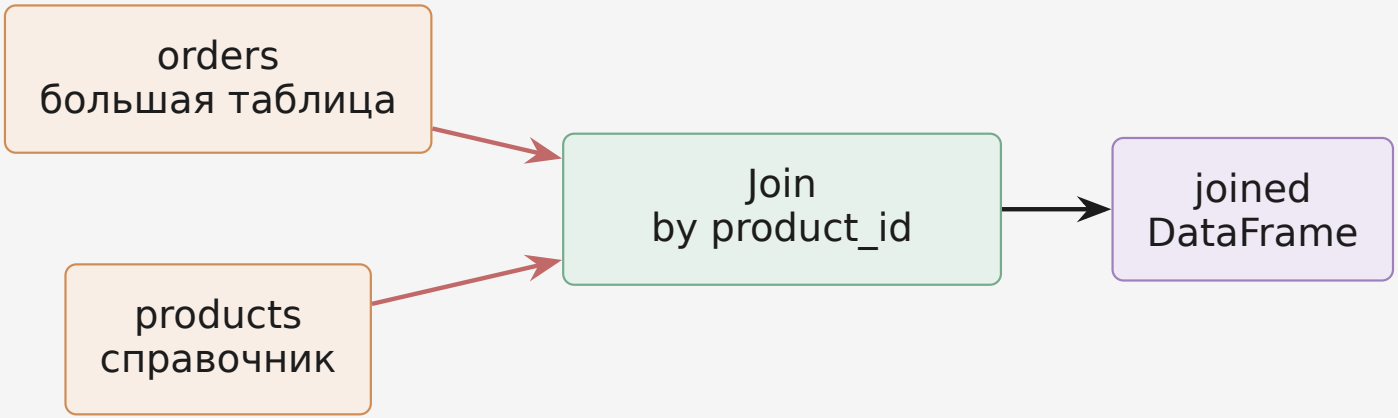
# Join-стратегии и повторное использование данных



# Join как распределённая операция

**Проблема**

Для соединения таблиц по ключу Spark должен обеспечить, чтобы строки с одинаковым ключом оказались совместно доступными одной задаче.




**Для оптимизации важно знать:**

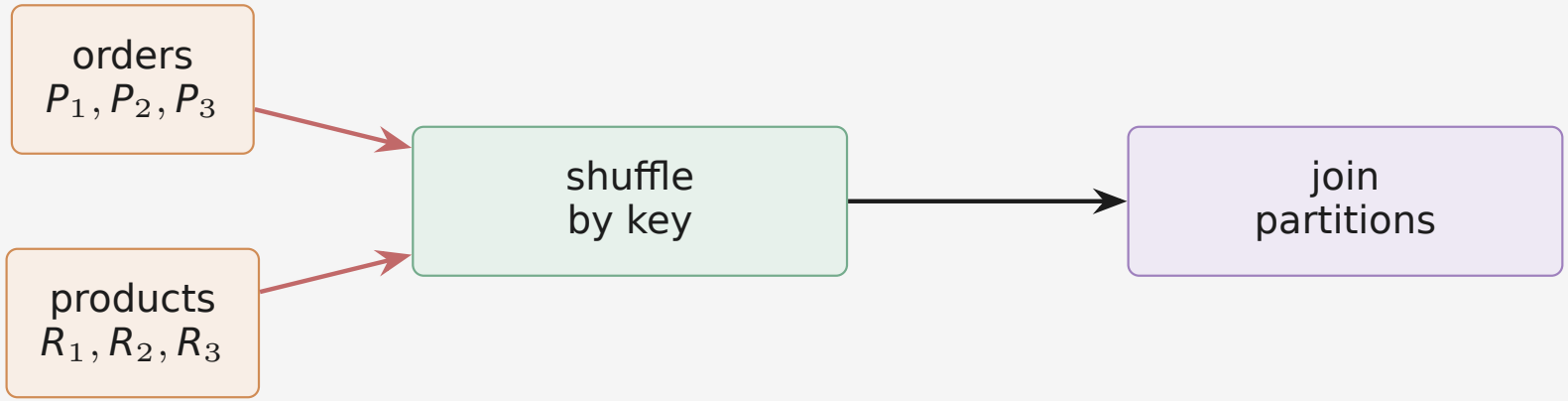
- размер каждой стороны;
- распределение ключей;
- требуется ли перемешивание;
- можно ли безопасно использовать broadcast.



# Shuffle join

**Определение**

**Shuffle join** — стратегия, при которой обе стороны соединения перераспределяются по ключу join.

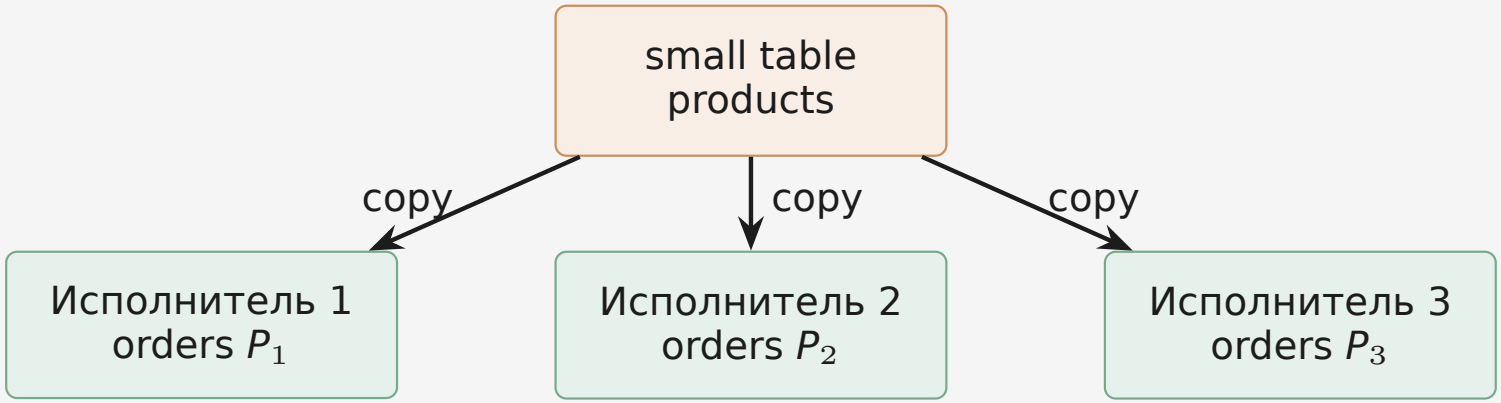



**Когда типично нужен.** Обе таблицы велики, маленькой стороны нет, ключи нужно физически сгруппировать.
**Цена:** сетевой обмен, запись/чтение промежуточных данных и возможный перекос.

# Broadcast join

**Определение**

**Broadcast join** — стратегия, при которой малая таблица копируется на исполнителей, а большая таблица обрабатывается без полного перемешивания по этой малой стороне.




$$|D_{small}| \ll |D_{large}|.$$

*Broadcast оправдан только тогда, когда малая сторона действительно помещается в память исполнителей.*

# Явное указание broadcast

```
from pyspark.sql.functions import broadcast

result = orders.join(
   broadcast(products),
   on="product_id",
   how="left"
)
```

**Ограничение**

Broadcast не является универсальным ускорителем. Если справочник уже не мал, его размножение по исполнителям создаёт давление на память и может ухудшить выполнение.

**Вопросы перед join:**

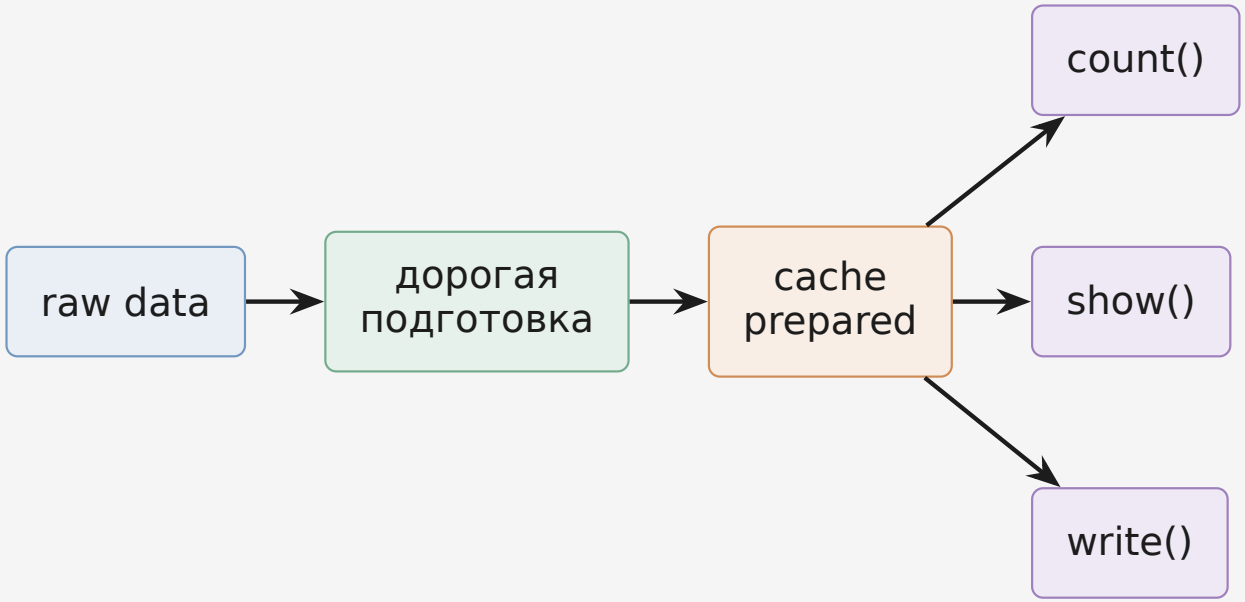
- насколько велика каждая сторона;
- как распределены ключи;
- какой план показывает `explain()`;
- возникает ли смещение по ключу соединения.



# Cache и persist

**Определение**

**Cache / persist** — механизмы сохранения вычисленных partitions для повторного использования в следующих actions.




память $\leftrightarrow$ стоимость повторного вычисления.



# Когда кэширование оправдано

```
prepared = (
    raw
    .join(reference, "key")
    .filter(...)
    .groupBy(...)
    .agg(...)
)

prepared = prepared.cache()

prepared.count() # материализация
prepared.show(20) # повторное
```

**Оправдано**

- результат дорог в построении;
- используется многократно;
- помещается в доступные ресурсы.

**Не оправдано**

- результат используется один раз;
- данные слишком велики;
- кэш вытесняет более важные данные.



Раздел

# Spark SQL и форматы хранения



# Spark SQL и DataFrame API

**Тезис**

Spark SQL и DataFrame API являются разными способами описать табличное вычисление, которое затем превращается в план Spark.




*Смысл SELECT, JOIN и GROUP BY сохраняется, но физическая модель выполнения становится распределённой.*



# Одна аналитика двумя способами

## DataFrame API

```
result_df = (
   sales
   .groupBy("category")
   .agg(F.sum("amount").alias("
        revenue"))
)
```

## Spark SQL

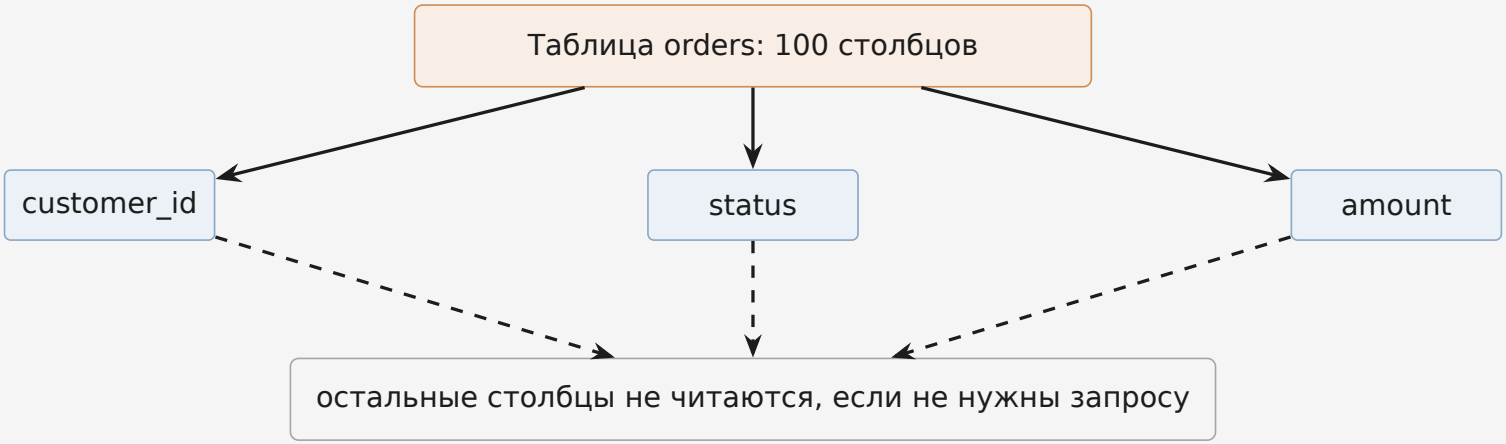
```
sales.createOrReplaceTempView("
     sales")

result_sql = spark.sql("""
   SELECT category,
          SUM(amount) AS revenue
   FROM sales
   GROUP BY category
""")
```

Оба выражения описывают один и тот же аналитический смысл. Далее Spark строит логический и физический план.



# Parquet как колоночный формат



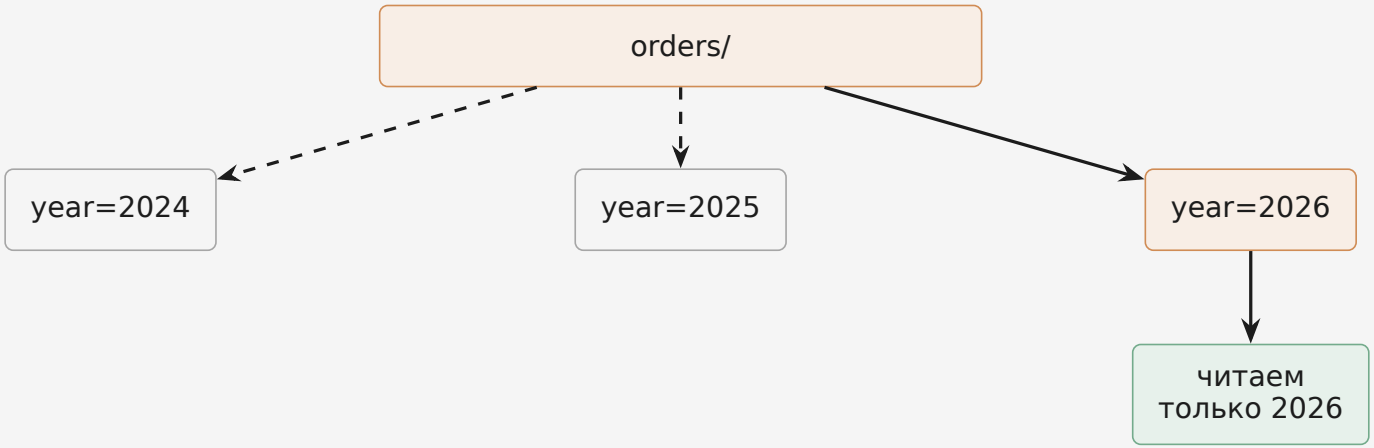

## Column pruning

Если запрос использует только несколько столбцов, Spark может не читать остальные столбцы из колоночного формата.



# Partition pruning

Физическая организация файлов может соответствовать значениям некоторого столбца.



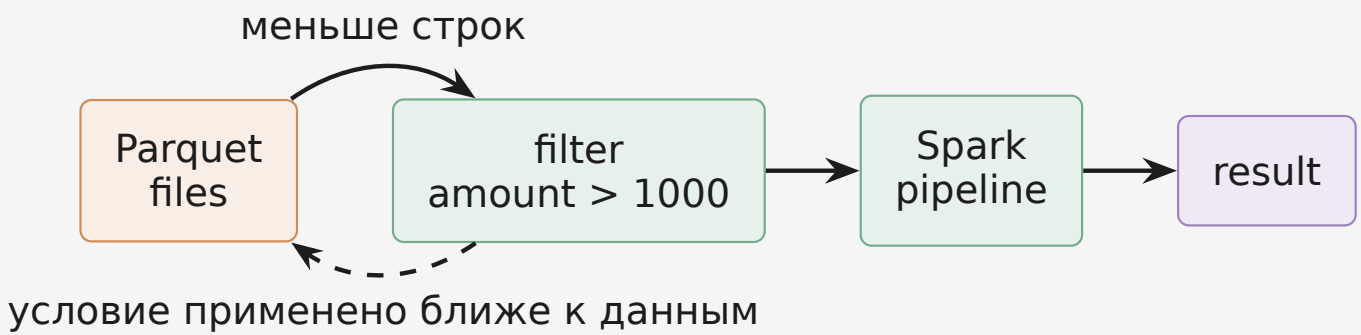

## Partition pruning

Если запрос содержит фильтр по partition-столбцу, Spark может исключить ненужные каталоги или файловые разделы из чтения.

# Predicate pushdown

**Определение**

**Predicate pushdown** — перенос условия фильтрации ближе к источнику данных, чтобы не передавать в pipeline заведомо ненужные строки.




**Общий принцип оптимизации.** Сначала уменьшить объём читаемых и перемещаемых данных, и только затем ускорять вычисления.



Раздел

# Сквозной поток вычислений



# Пример. Чтение и подготовка

```
orders = spark.read.parquet("data/orders")
products = spark.read.parquet("data/products")

orders_small = (
   orders
   .filter("status = 'paid'")
   .select("product_id", "quantity", "price")
)

products_small = products.select("product_id", "category")
```

**Смысл**

Рано сокращаем число строк и столбцов с помощью фильтрации, прореживания столбцов и получаем меньший объём данных для последующих стадий.



# Пример. Join и агрегация

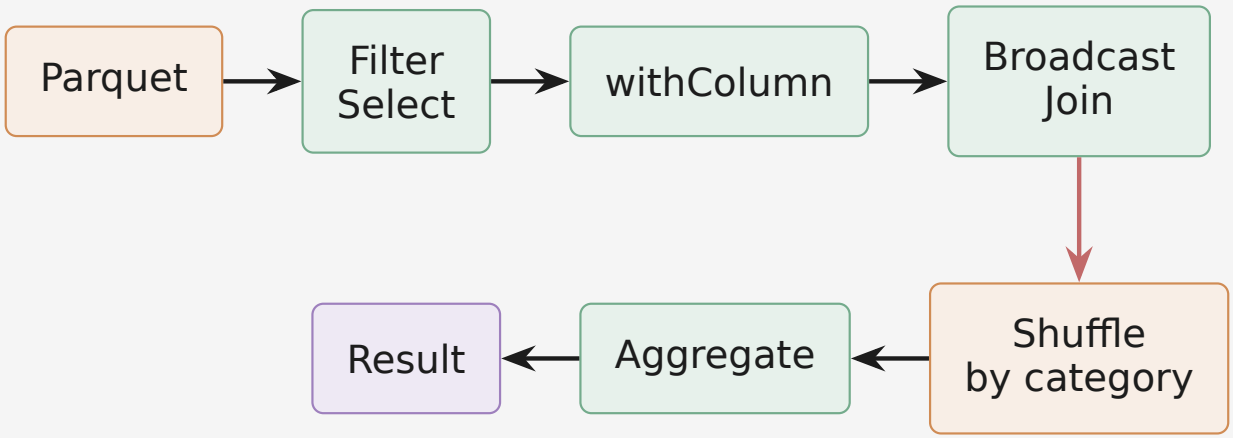
```
orders_amount = orders_small.withColumn(
   "amount",
   F.col("quantity") * F.col("price")
)

joined = orders_amount.join(
   broadcast(products_small),
   on="product_id",
   how="left"
)

revenue_by_category = (
   joined
   .groupBy("category")
   .agg(F.sum("amount").alias("revenue"))
)
```



# Физическая логика сквозного потока обработки




**Чтение кода по-Spark-овски:**

- какие операции уменьшают данные рано;
- где операции внутренние;
- где возникает join;
- где возникает перемешивание;
- какое действие запустит выполнение.



# Проверка плана и запуск

```
revenue_by_category.explain()

revenue_by_category.show()

revenue_by_category.write.mode("overwrite").parquet(
   "out/revenue_by_category"
)
```

**Практическая привычка**

Перед оптимизацией Spark-задачи нужно увидеть план. Отследить какие стадии построены, где перемешивание, какой тип join выбран, какие столбцы читаются и каков ожидаемый путь данных.



# Типичные ошибки

- считать Spark «быстрым Pandas»;
- ожидать линейного ускорения от увеличения числа ядер;
- полагать, что больше partitions всегда лучше;
- использовать `collect()` как обычный способ просмотра данных;
- кэшировать каждый промежуточный DataFrame;
- не отличать логический смысл `groupBy` от физического перемешивания.

**Главная коррекция мышления**

Spark-программа оценивается не только по формулам и API-вызовам, но и по тому, как данные читаются, делятся, перемещаются и материализуются.



# Алгоритм диагностики Spark-задачи



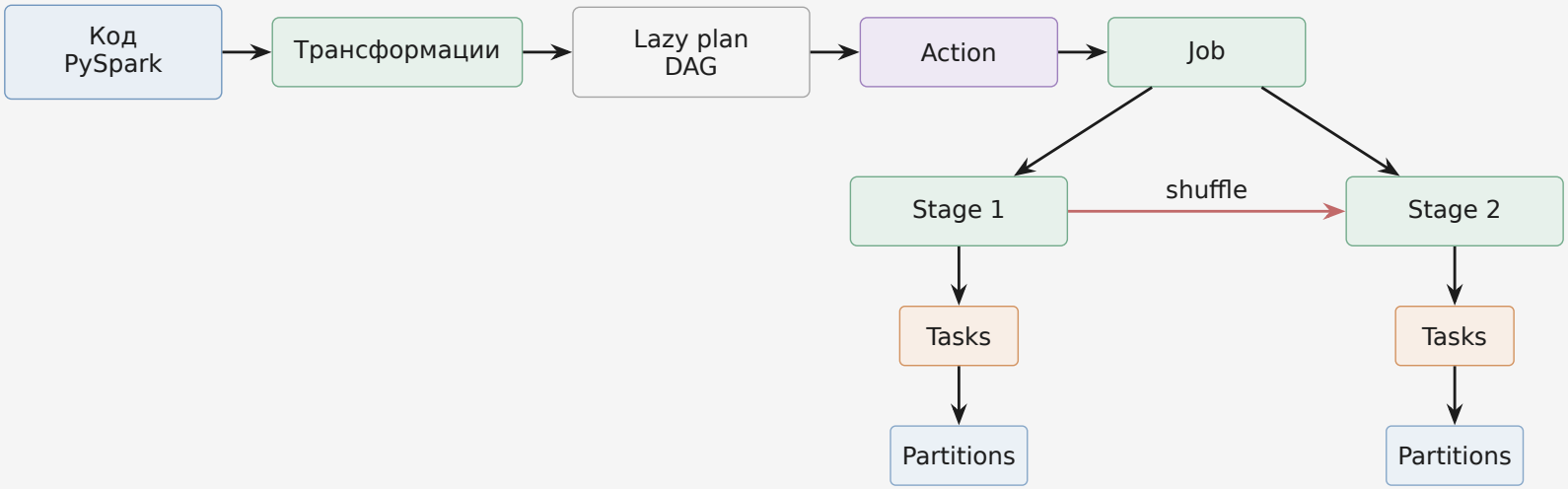

*Оптимизация начинается не с набора конфигурационных параметров, а с понимания пути данных.*

Раздел

# Итоги



# Итоговая модель Spark-вычисления




## Смысл схемы

Пользователь описывает преобразования; действие требует результата; Spark строит job, делит его на стадии, выполняет задачи над партициями и при необходимости запускает перемешиание данных между стадиями.



# Что студент должен уметь после лекции

Студент должен уметь по фрагменту PySpark-кода объяснить:

- какие операции являются трансформациями и действиями;
- где Spark только строит план, а где начинается выполнение;
- как действие порождает job;
- как job разбивается на стадии и задачи;
- как партиции связаны с параллелизмом;
- где вероятно перемецивание;
- чем внутренние-преобразования отличаются от широких-преобразований;
- когда broadcast join может быть полезен;
- почему кэш не является универсальной оптимизацией;
- зачем смотреть `explain()`.



# Спасибо за внимание!

